Part 3. 고급 주제 — 시뮬레이션, 강화학습, 휴머노이드

목차

1. 개요
2. Sim-to-Real Transfer
3. 강화학습 파인튜닝
4. 휴머노이드 제어
5. 월드 모델
6. 데이터 스케일링
7. 멀티모달 로봇 학습
8. 안전과 윤리
9. PyTorch 구현 예시
10. 참고 문헌

1. 개요

1.1 Part 3에서 다루는 내용

이 문서는 로봇 AI의 **고급 주제**들을 다룹니다:

- **Sim-to-Real**: 시뮬레이션에서 실제 환경으로의 전이
- **강화학습**: BC 모델의 RL 파인튜닝
- **휴머노이드**: 전신 제어와 균형
- **월드 모델**: 환경 예측 및 계획
- **데이터 스케일링**: 효율적 데이터 활용
- **안전**: 물리적 제약과 윤리

1.2 사전 지식 요구사항

- Part 1, 2의 내용 (VAE, Diffusion, VLA 모델)
- 강화학습 기초: MDP, Policy Gradient, Value Function
- 로봇공학 기초: Forward/Inverse Kinematics, Dynamics
- 제어 이론: PID, MPC

2. Sim-to-Real Transfer

시뮬레이션에서 학습한 정책을 실제 로봇에 적용하는 것은 로봇 AI의 핵심 과제입니다.

2.1 Domain Gap의 수학적 정의

시뮬레이션 분포 $p_{\text{sim}}(s, a, s')$과 **실제 분포** $p_{\text{real}}(s, a, s')$의 차이:

$$D_{\text{TV}}(p_{\text{sim}}, p_{\text{real}}) = \frac{1}{2} \int |p_{\text{sim}}(x) - p_{\text{real}}(x)| dx$$

또는 KL Divergence:

$$D_{KL}(p_{\text{sim}} \| p_{\text{real}}) = \mathbb{E}_{x \sim p_{\text{sim}}} \left[\log \frac{p_{\text{sim}}(x)}{p_{\text{real}}(x)} \right]$$

Domain Gap의 원인:

1. **물리 파라미터 불일치**: 마찰, 질량, 관성
2. **센서 노이즈**: 시뮬레이션은 완벽, 실제는 노이즈 존재
3. **시각적 차이**: 렌더링 vs 실제 이미지
4. **동역학 단순화**: 접촉, 변형 등

2.2 Domain Randomization

핵심 아이디어

시뮬레이션 파라미터 ξ를 **랜덤하게 변화**시켜, 정책이 **다양한 조건**에 강건하도록:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\xi \sim p(\xi)} \left[\mathbb{E}_{\tau \sim \pi, \mathcal{M}_\xi} \left[\sum_t \gamma^t r_t \right] \right]$$

여기서:

- $\mathcal{M}_\xi$: 파라미터 ξ를 가진 시뮬레이터
- $p(\xi)$: 파라미터 분포

랜덤화 대상

물리 파라미터:

- 마찰 계수: $\mu \sim \text{Uniform}(0.5, 1.5)$
- 질량: $m \sim \text{Uniform}(0.8m_0, 1.2m_0)$
- 관성: $I \sim \text{Uniform}(0.9I_0, 1.1I_0)$
- 댐핑: $b \sim \text{Uniform}(0.5b_0, 1.5b_0)$

시각 파라미터:

- 조명 강도: $L \sim \text{Uniform}(0.5, 2.0)$
- 조명 위치: $(x, y, z) \sim \text{Uniform}(\text{범위})$
- 텍스처: 랜덤 교체
- 카메라 위치/각도: 노이즈 추가

동역학 파라미터:

- 액추에이터 지연: $\Delta t \sim \text{Uniform}(0, 50\text{ms})$
- 센서 노이즈: $n \sim \mathcal{N}(0, \sigma^2)$
- 제어 주파수: 랜덤 드롭

이론적 근거

정책이 충분히 다양한 ξ에서 학습되면, 실제 환경 ξ_{real}도 해당 분포 내에 포함될 가능성 증가:

$$\xi_{\text{real}} \in \text{support}(p(\xi))$$

실험적 검증 (OpenAI, 2019):

- Rubik's Cube 해결 로봇
- 시뮬레이션에서만 학습 (DR 적용)
- 실제 로봇에서 성공률 ~60%

2.3 Domain Adaptation

Adversarial Domain Adaptation

목표: Feature extractor f_θ가 **도메인 불변** 특징 추출.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} - \lambda \mathcal{L}_{\text{domain}}$$

Task Loss (정책 학습):

$$\mathcal{L}_{\text{task}} = \mathbb{E}_{(s,a)\sim\mathcal{D}_{\text{sim}}}[-\log \pi_\theta(a \mid f_\theta(s))]$$

Domain Loss (도메인 분류):

Domain classifier D_ϕ:

$$\mathcal{L}_{\text{domain}} = \mathbb{E}_{s\sim\mathcal{D}_{\text{sim}}}[\log D_\phi(f_\theta(s))] + \mathbb{E}_{s\sim\mathcal{D}_{\text{real}}}[\log(1 - D_\phi(f_\theta(s)))]$$

Gradient Reversal Layer:

Forward에서는 $f_\theta(s)$ 그대로, Backward에서는 그래디언트 부호 반전:

$$\frac{\partial \mathcal{L}_{\text{domain}}}{\partial \theta} \to -\lambda \frac{\partial \mathcal{L}_{\text{domain}}}{\partial \theta}$$

f_θ는 도메인 분류를 **어렵게** 만들려고 학습 → 도메인 불변 특징.

Cycle-Consistency

Sim2Real + Real2Sim:

$$\mathcal{L}_{\text{cycle}} = \mathbb{E}[\|G_{\text{real2sim}}(G_{\text{sim2real}}(x_{\text{sim}})) - x_{\text{sim}}\|]$$

CycleGAN 방식으로 이미지 변환.

2.4 Sim-to-Real 성공 사례

프로젝트	기법	성과
OpenAI Rubik's Cube	DR (광범위)	실제 로봇 60% 성공
Boston Dynamics Atlas	Model-based + Real Data	파쿠르, 백플립
Tesla Optimus	Sim2Real + Imitation	물체 분류, 조작
Figure 01	VLA (OpenVLA)	일상 태스크

---

3. 강화학습 파인튜닝

Behavior Cloning으로 학습한 정책을 강화학습으로 개선.

3.1 BC의 한계

Distributional Shift:

학습 데이터 $\mathcal{D}_{\text{expert}}$와 실행 시 방문 상태 $\mathcal{D}_\pi$의 불일치:

$$D_{\text{TV}}(\mathcal{D}_{\text{expert}}, \mathcal{D}_\pi) > 0$$

작은 오차가 누적 → 미지의 상태 도달 → 실패.

이론적 분석 (Ross et al., 2011):

BC 정책의 기대 비용:

$$J(\pi_{\text{BC}}) \leq J(\pi^*) + O(T^2\epsilon)$$

T: 시간 스텝, ϵ: BC 오차

시간이 길수록 오차가 **제곱**으로 증가!

3.2 BC + RL 하이브리드

KL-Constrained RL

BC 정책 π_{BC}에서 크게 벗어나지 않도록:

$$\pi_{k+1} = \arg\max_\pi \mathbb{E}_{\tau\sim\pi}[R(\tau)] - \beta D_{KL}(\pi\|\pi_{\text{BC}})$$

장점:

- BC 정책의 안전성 유지
- RL의 탐색으로 성능 향상

단점:

- β 튜닝 필요
- BC 정책이 나쁘면 제약 역효과

Residual RL

BC 정책 위에 **잔차(residual)** 정책 학습:

$$a = \pi_{\text{BC}}(s) + \pi_{\text{residual}}(s)$$

π_{residual}는 작은 보정만 학습 → 안정적.

3.3 PPO (Proximal Policy Optimization)

목적 함수

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

여기서:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$: probability ratio
- $\hat{A}_t$: advantage estimate (GAE)
- $\epsilon = 0.2$: clipping parameter

Advantage 계산 (GAE)

Generalized Advantage Estimation:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$\lambda \in [0, 1]$: bias-variance 트레이드오프

- $\lambda = 0$: low variance, high bias (TD)
- $\lambda = 1$: high variance, low bias (MC)

PPO 알고리즘

```
for iteration in range(N):
    # Collect trajectories
    trajectories = collect_trajectories(π_θ_old)

    # Compute advantages
    advantages = compute_gae(trajectories)

    # Update policy (multiple epochs)
    for epoch in range(K):
        for batch in minibatch(trajectories):
            # Compute ratio
            ratio = π_θ(a|s) / π_θ_old(a|s)

            # Clipped objective
            L_clip = min(ratio * A, clip(ratio, 1-ε, 1+ε) * A)

            # Value loss
            L_vf = (V_θ(s) - V_target)²

            # Entropy bonus
            L_ent = -H(π_θ(·|s))

            # Total loss
            L = -L_clip + c₁*L_vf - c₂*L_ent

            # Update
            θ ← θ - η▽L

    θ_old ← θ
```

3.4 Reward Shaping

희소 보상 문제 해결.

Potential-Based Reward Shaping

$$r'(s, a, s') = r(s, a, s') + \gamma\Phi(s') - \Phi(s)$$

$\Phi : \mathcal{S} \to \mathbb{R}$: potential function

이론적 보장 (Ng et al., 1999):

Potential-based shaping은 **최적 정책을 변경하지 않음**:

$$\pi^* = \arg\max_\pi \mathbb{E}[R] = \arg\max_\pi \mathbb{E}[R']$$

예시

목표까지 거리:

$$\Phi(s) = -\|s_{\text{gripper}} - s_{\text{goal}}\|$$

서브골 도달:

$$\Phi(s) = \begin{cases} 1 & \text{if subgoal reached} \\ 0 & \text{otherwise} \end{cases}$$

3.5 Offline RL

시뮬레이션 없이 **고정 데이터셋**에서만 학습.

Conservative Q-Learning (CQL)

Q-value를 **보수적**으로 추정:

$$\mathcal{L}_{\text{CQL}} = \alpha \mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_a \exp(Q(s,a)) - \mathbb{E}_{a\sim\mathcal{D}}[Q(s,a)] \right] + \mathcal{L}_{\text{Bellman}}$$

효과: Out-of-distribution 행동의 Q-value를 낮게 추정 → 안전한 정책.

---

4. 휴머노이드 제어

4.1 전신 동역학

방정식

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau + J^\top F_{\text{ext}}$$

여기서:

- $q \in \mathbb{R}^n$: 일반화 좌표 (관절 각도)
- $M(q) \in \mathbb{R}^{n\times n}$: 관성 행렬
- $C(q,\dot{q}) \in \mathbb{R}^{n\times n}$: 코리올리/원심력 행렬
- $G(q) \in \mathbb{R}^n$: 중력 벡터
- $\tau \in \mathbb{R}^n$: 관절 토크
- $J \in \mathbb{R}^{6\times n}$: 자코비안
- $F_{\text{ext}} \in \mathbb{R}^6$: 외력

예시: 휴머노이드 자유도

- 양팔: $7 \times 2 = 14$ DOF
- 양다리: $6 \times 2 = 12$ DOF
- 몸통/머리: $3 + 2 = 5$ DOF

- **총 31 DOF**

Floating base (6 DOF) 포함 시 **37 DOF**.

4.2 균형 제어

Zero Moment Point (ZMP)

정의: 지면 반력의 합력 모멘트가 0이 되는 지점.

$$\text{ZMP} = \frac{\sum_i m_i(g + \ddot{z}_i)[x_i, y_i]^\top}{\sum_i m_i(g + \ddot{z}_i)}$$

균형 조건:

$$\text{ZMP} \in \text{Support Polygon}$$

Support Polygon: 접촉점들의 convex hull.

Center of Mass (CoM) 동역학

$$m\ddot{\mathbf{r}}_{\text{CoM}} = \sum_i \mathbf{F}_i + m\mathbf{g}$$

$$\dot{\mathbf{L}} = \sum_i \mathbf{r}_i \times \mathbf{F}_i$$

$\mathbf{L}$: 각운동량

Linear Inverted Pendulum Model (LIPM):

CoM을 단일 질점으로 근사:

$$\ddot{x} = \frac{g}{z_{\text{CoM}}}(x - x_{\text{ZMP}})$$

간단한 모델로 보행 패턴 생성.

4.3 보행 생성

Foot Placement

다음 발 위치 계산:

$$\mathbf{p}_{\text{foot}} = \mathbf{p}_{\text{CoM}} + \frac{\dot{\mathbf{p}}_{\text{CoM}}}{\omega}$$

$$\omega = \sqrt{\frac{g}{z_{\text{CoM}}}}$$

Capture Point 개념.

Trajectory Optimization

목적 함수:

$$\min_{\mathbf{q}(t),\mathbf{u}(t)} \int_0^T \left(\|\mathbf{q}(t) - \mathbf{q}_{\text{ref}}(t)\|_Q^2 + \|\mathbf{u}(t)\|_R^2 \right) dt$$

제약:

- 동역학: $M\ddot{q} + C\dot{q} + G = \tau$
- 접촉: $\text{ZMP} \in \text{polygon}$
- 관절 한계: $q_{\min} \leq q \leq q_{\max}$
- 토크 한계: $|\tau| \leq \tau_{\max}$

4.4 Model Predictive Control (MPC)

실시간 최적화:

$$\min_{u_{0:N-1}} \sum_{k=0}^{N-1} \left[(x_k - x_{\text{ref}})^\top Q (x_k - x_{\text{ref}}) + u_k^\top R u_k \right]$$

제약:

- $x_{k+1} = f(x_k, u_k)$
- $x_k \in \mathcal{X}$, $u_k \in \mathcal{U}$

Receding Horizon:

매 시간 스텝에서 재계산 → 첫 번째 제어만 적용 → 반복.

구현

```
for t in range(T):
    # Current state
    x_current = get_state()

    # Solve optimization
    u_optimal = solve_mpc(x_current, x_ref, N)

    # Apply first control
    apply_control(u_optimal[0])

    # Update state
    x_current = step_dynamics(x_current, u_optimal[0])
```

4.5 Whole-Body Control

전신을 동시에 제어하는 계층적 접근.

Task Space Control

End-effector 목표 $\mathbf{x}_d$:

$$\mathbf{x} = f_{\text{fk}}(\mathbf{q})$$

역기구학:

$$\dot{\mathbf{x}} = J(\mathbf{q})\dot{\mathbf{q}}$$

$$\dot{\mathbf{q}} = J^{\dagger}\dot{\mathbf{x}}_d$$

$J^{\dagger}$: pseudo-inverse

Hierarchical QP

우선순위별 태스크:

1. **균형** (최우선)
2. **End-effector 위치**
3. **자세 유지**
4. **에너지 최소화**

Null space projection으로 해결.

---

5. 월드 모델

환경을 예측하는 모델 학습 → 시뮬레이션에서 계획.

5.1 Model-Based RL

기본 프레임워크

1. **환경 상호작용**: 데이터 수집 $\mathcal{D} = \{(s_t, a_t, s_{t+1})\}$
2. **모델 학습**: $\hat{s}_{t+1} = f_\theta(s_t, a_t)$
3. **정책 학습**: 모델에서 시뮬레이션하여 정책 업데이트
4. **반복**

장점:

- Sample efficiency 향상
- 안전한 탐색

단점:

- Model bias
- Compounding error

5.2 Dreamer 아키텍처

RSSM (Recurrent State-Space Model)

Deterministic path (RNN):

$$h_t = f_{\text{RNN}}(h_{t-1}, z_{t-1}, a_{t-1})$$

Stochastic path (latent):

$$z_t \sim p_\theta(z_t \mid h_t, o_t)$$

Decoder:

$$\hat{o}_t \sim p_\theta(o_t \mid h_t, z_t)$$

Reward/Value predictor:

$$\hat{r}_t = p_\theta(r_t \mid h_t, z_t)$$

$$\hat{v}_t = V_\theta(h_t, z_t)$$

학습 목표

Representation learning:

$$\mathcal{L}_{\text{rep}} = \mathbb{E}\left[-\log p(o_t \mid h_t, z_t) + \beta D_{KL}(p(z_t \mid h_t, o_t) \| p(z_t \mid h_t))\right]$$

Dynamics learning:

$$\mathcal{L}_{\text{dyn}} = \mathbb{E}[-\log p(z_t \mid h_t)]$$

Imagination

잠재 공간에서 정책 학습:

```
for imagination_step in range(H):
    # Sample action from policy
    a_t = π(h_t, z_t)

    # Predict next state
    h_{t+1} = f_RNN(h_t, z_t, a_t)
    z_{t+1} ~ p(z_{t+1} | h_{t+1})

    # Predict reward
    r_{t+1} = p(r_{t+1} | h_{t+1}, z_{t+1})
```

이 imagined trajectory로 정책 업데이트.

5.3 NVIDIA Cosmos

특징

- **Video World Model**: 비디오 입력/출력
- **Physical Simulation**: 물리 법칙 암묵적 학습
- **최대 30초** 물리 시뮬레이션
- **다양한 조건**: 텍스트, 이미지, 비디오

아키텍처 (추정)

```
[Video Tokenizer]
비디오 프레임 → VAE → 잠재 토큰

[Transformer Backbone]
조건 (텍스트/이미지) + 잠재 토큰 → 다음 토큰 예측

[Video Decoder]
잠재 토큰 → VAE Decoder → 비디오 프레임
```

물리 일관성:

- 중력, 관성 등을 데이터에서 암묵적 학습
- Trajectory consistency loss
- Multi-view consistency

로봇 응용

- **정책 시각화**: 행동의 결과 예측
- **Counterfactual reasoning**: "만약 ~했다면?"
- **안전 검증**: 실행 전 시뮬레이션

---

6. 데이터 스케일링

6.1 스케일링 법칙

Chinchilla Scaling Law

$$\mathcal{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

여기서:

- N: 모델 파라미터 수
- D: 데이터 크기 (토큰 수)
- E: irreducible loss

- α, β: 스케일링 지수

로봇 도메인:

- $\alpha \approx 0.34$ (Scaling Up 논문)
- $\beta \approx 0.28$

최적 할당:

$$N \propto D^{\beta/\alpha}$$

모델 크기와 데이터 크기를 **균형있게** 증가.

6.2 데이터 혼합

π0의 발견

혼합 비율이 성능 결정:

$$\mathcal{L}_{\text{total}} = \sum_{i=1}^{K} \lambda_i \mathcal{L}_i$$

λ_i: 데이터셋 i의 가중치

실험 결과:

- 단순 균등 혼합: 성능 낮음
- 최적 혼합 (탐색): 성능 2배 향상

최적화 방법

Bayesian Optimization:

$$\lambda^* = \arg\min_{\lambda} \mathcal{L}_{\text{val}}(\text{train}(\lambda))$$

Grid Search (소규모):

모든 조합 시도.

Evolutionary:

유전 알고리즘으로 탐색.

6.3 Data Quality vs Quantity

특성	High Quality	High Quantity
수집 비용	높음	낮음

다양성	낮음	높음
성공률	높음	중간
적합 모델	BC	BC + RL

결론: Quality와 Quantity의 **균형** 필요.

6.4 Co-training

여러 태스크를 **동시에** 학습:

$$\mathcal{L}_{\text{total}} = \sum_{i=1}^{T} w_i \mathcal{L}_{\text{task}_i}$$

Positive Transfer:

- 관련 태스크 간 지식 공유
- 예: "물체 잡기" + "물체 놓기"

Negative Transfer:

- 무관한 태스크 간 간섭
- 예: "물체 잡기" + "걷기"

해결책: Task-specific heads + shared backbone.

---

7. 멀티모달 로봇 학습

7.1 Vision + Proprioception

Sensor Fusion

Early Fusion:

$$h = f_\theta([v; p])$$

v: vision, p: proprioception

Late Fusion:

$$h_v = f_v(v), \quad h_p = f_p(p)$$

$$h = f_{\text{fuse}}([h_v; h_p])$$

Cross-Attention Fusion:

$$h = \text{CrossAttn}(h_v, h_p, h_p)$$

실험 비교

방법	성능	계산 비용
Vision only	낮음	낮음
Proprio only	중간	매우 낮음
Early Fusion	중간	중간
Late Fusion	**높음**	중간
Cross-Attention	최고	높음

7.2 Language Grounding

CLIP for Robotics

CLIP 임베딩으로 언어-이미지 정렬:

$$\text{sim}(I, T) = \frac{f_{\text{image}}(I)^\top f_{\text{text}}(T)}{\|f_{\text{image}}(I)\| \|f_{\text{text}}(T)\|}$$

Object Detection:

$$\text{score}(bbox) = \text{sim}(\text{crop}(I, bbox), T)$$

Reward Shaping:

$$r(s, g) = \text{sim}(s_{\text{image}}, g_{\text{text}})$$

7.3 Audio

Sound-based Feedback:

- 접촉 감지: 소리로 물체 재질 파악
- 상태 추정: 부서짐, 넘어짐 등

---

8. 안전과 윤리

8.1 물리적 안전 제약

Hard Constraints

관절 한계:

$$q_{\min} \leq q \leq q_{\max}$$

속도 한계:

$$\|\dot{q}\| \leq v_{\max}$$

토크 한계:

$$\|\tau\| \leq \tau_{\max}$$

Safety Layer

정책 출력을 **필터링**:

$$a_{\text{safe}} = \arg\min_{a \in \mathcal{A}_{\text{safe}}} \|a - a_\pi\|$$

여기서:

$$\mathcal{A}_{\text{safe}} = \{a : \text{constraints satisfied}\}$$

구현: QP solver

$$\min_a \|a - a_\pi\|^2$$

subject to: safety constraints

8.2 Human-in-the-Loop

Intervention Rate

사람이 개입하는 빈도:

$$\rho = \frac{\text{개입 횟수}}{\text{총 스텝}}$$

π0 실험:

- 2초 간격 개입: 성공률 **95%**
- 완전 자율: 성공률 60%

Shared Autonomy:

$$a = \alpha a_{\text{human}} + (1 - \alpha) a_{\text{robot}}$$

$\alpha \in [0, 1]$: 사람의 제어 정도

Learning from Interventions

사람의 개입을 **데이터**로:

$$\mathcal{D}_{\text{intervention}} = \{(s, a_{\text{robot}}, a_{\text{human}})\}$$

이를 활용하여 정책 개선 (DAgger).

8.3 윤리적 고려사항

로봇의 행동 원칙:

1. 사람에게 해를 끼치지 않음
2. 사람의 명령을 따름 (1번과 충돌하지 않는 한)
3. 자기 보존 (1, 2번과 충돌하지 않는 한)

데이터 편향:

- 수집 데이터가 특정 그룹에 편향되면 정책도 편향
- 다양성 확보 필요

투명성:

- 로봇의 결정 과정 설명 가능해야 함
- Attention visualization, saliency maps

---

9. PyTorch 구현 예시

9.1 PPO 구현

```python
import torch
import torch.nn as nn
import torch.optim as optim

class PPO:
    def __init__(self, policy_net, value_net, lr=3e-4, gamma=0.99,
                 lam=0.95, clip_epsilon=0.2, epochs=10):
        self.policy = policy_net
        self.value = value_net

        self.optimizer_policy = optim.Adam(self.policy.parameters(),
        self.optimizer_value = optim.Adam(self.value.parameters(), l

        self.gamma = gamma
        self.lam = lam
        self.clip_epsilon = clip_epsilon
        self.epochs = epochs

    def compute_gae(self, rewards, values, dones):
        """Generalized Advantage Estimation"""
        advantages = []
        gae = 0

        for t in reversed(range(len(rewards))):
            if t == len(rewards) - 1:
                next_value = 0
            else:
                next_value = values[t + 1]

            delta = rewards[t] + self.gamma * next_value * (1 - don
            gae = delta + self.gamma * self.lam * (1 - dones[t]) * g
            advantages.insert(0, gae)

        return torch.tensor(advantages)

    def update(self, states, actions, old_log_probs, returns, advant
        """PPO update"""
        for _ in range(self.epochs):
            # Compute current log probs
            new_log_probs, entropy = self.policy(states, actions)

            # Compute ratio
            ratio = torch.exp(new_log_probs - old_log_probs)

            # Clipped surrogate objective
            surr1 = ratio * advantages
            surr2 = torch.clamp(ratio, 1 - self.clip_epsilon,
                                1 + self.clip_epsilon) * advantages
            policy_loss = -torch.min(surr1, surr2).mean()

            # Value loss
            values = self.value(states)
```

```
value_loss = 0.5 * (returns - values).pow(2).mean()

# Total loss
loss = policy_loss + 0.5 * value_loss - 0.01 * entropy.r

# Update
self.optimizer_policy.zero_grad()
self.optimizer_value.zero_grad()
loss.backward()
self.optimizer_policy.step()
self.optimizer_value.step()
```

9.2 Dreamer RSSM

```python
class RSSM(nn.Module):
    def __init__(self, obs_dim, action_dim, hidden_dim=200, latent_
        super().__init__()

        # Deterministic path (RNN)
        self.rnn = nn.GRUCell(latent_dim + action_dim, hidden_dim)

        # Stochastic path (encoder)
        self.encoder = nn.Sequential(
            nn.Linear(hidden_dim + obs_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 2 * latent_dim)  # mean, std
        )

        # Prior
        self.prior = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 2 * latent_dim)
        )

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(hidden_dim + latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, obs_dim)
        )

        # Reward predictor
        self.reward_pred = nn.Sequential(
            nn.Linear(hidden_dim + latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, obs, action, h_prev, z_prev):
        # Deterministic update
        h = self.rnn(torch.cat([z_prev, action], dim=-1), h_prev)

        # Prior
        prior_params = self.prior(h)
        prior_mean, prior_std = torch.chunk(prior_params, 2, dim=-1
        prior_std = F.softplus(prior_std) + 0.1

        # Posterior
        post_params = self.encoder(torch.cat([h, obs], dim=-1))
        post_mean, post_std = torch.chunk(post_params, 2, dim=-1)
        post_std = F.softplus(post_std) + 0.1

        # Sample latent
        z = post_mean + post_std * torch.randn_like(post_std)
```

```
        # Decode
        obs_pred = self.decoder(torch.cat([h, z], dim=-1))

        # Reward
        reward_pred = self.reward_pred(torch.cat([h, z], dim=-1))

        return obs_pred, reward_pred, h, z, (prior_mean, prior_std)
```

9.3 Safety Layer (QP)

```python
import cvxpy as cp

class SafetyLayer:
    def __init__(self, q_min, q_max, dq_max, tau_max):
        self.q_min = q_min
        self.q_max = q_max
        self.dq_max = dq_max
        self.tau_max = tau_max

    def filter_action(self, q_current, dq_current, action_desired):
        """
        Solve QP to find safe action closest to desired action
        """
        n = len(action_desired)

        # Decision variable
        action = cp.Variable(n)

        # Objective: minimize distance to desired action
        objective = cp.Minimize(cp.sum_squares(action - action_desi

        # Constraints
        constraints = []

        # Joint limits (assuming action is acceleration)
        q_next = q_current + dq_current * dt + 0.5 * action * dt**2
        constraints.append(q_next >= self.q_min)
        constraints.append(q_next <= self.q_max)

        # Velocity limits
        dq_next = dq_current + action * dt
        constraints.append(cp.abs(dq_next) <= self.dq_max)

        # Torque limits
        constraints.append(cp.abs(action) <= self.tau_max)

        # Solve
        problem = cp.Problem(objective, constraints)
        problem.solve()

        if problem.status == 'optimal':
            return action.value
        else:
            # If infeasible, return zero action (emergency stop)
            return np.zeros(n)
```

---

10. 참고 문헌

1. **Domain Randomization**: Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", IROS 2017
2. **PPO**: Schulman et al., "Proximal Policy Optimization Algorithms", 2017
3. **Dreamer**: Hafner et al., "Dream to Control: Learning Behaviors by Latent Imagination", ICLR 2020
4. **GAE**: Schulman et al., "High-Dimensional Continuous Control Using Generalized Advantage Estimation", ICLR 2016
5. **CQL**: Kumar et al., "Conservative Q-Learning for Offline Reinforcement Learning", NeurIPS 2020
6. **Isaac Gym**: Makoviychuk et al., "Isaac Gym: High Performance GPU-Based Physics Simulation", NeurIPS 2021
7. **Cosmos**: NVIDIA, "Cosmos World Foundation Model", 2025
8. **Scaling Laws**: Kaplan et al., "Scaling Laws for Neural Language Models", 2020
9. **Safety**: García et al., "A Comprehensive Survey on Safe Reinforcement Learning", JMLR 2015
10. **Humanoid Control**: Atkeson et al., "Dynamic Balance Force Control for Compliant Humanoid Robots", ICRA 2006

다음 파트: Part 4. 실습 가이드 — PyTorch 구현과 학습 팁